

Utilizando *operadores*

Operadores são os símbolos que fazem contas, comparações e combinações de condições. Nesta aula: os aritméticos (com a divisão inteira e o resto, que aparecem em muito problema), a ordem de precedência, comparações, `and`, `or`, `not`, atribuição composta, `in` e `is`.

+ - * / // % **

Precedência

== != < >

and · or · not

+= -=

in · is

Formação Claude Code 2026 · Baseado em docs.python.org (Expressions: operator precedence; Built-in Types: numeric operations, comparisons, boolean operations), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em [exemplos/](#) · Saídas reais

As sete *operações*

aritmeticos.py

```
a, b = 17, 5
print(a + b)      # soma
print(a - b)      # subtração
print(a * b)      # multiplicação
print(a / b)      # divisão (sempre float)
print(a // b)     # divisão inteira
print(a % b)      # resto da divisão
print(a ** 2)     # potência
print(divmod(a, b)) # (divisão inteira, resto) de uma vez
```

SAÍDA

```
22
12
85
3.4
3
2
289
(3, 2)
```

OPERADOR	17 E 5
/	3.4 (divisão "normal")
//	3 (quantas vezes o 5 cabe)
%	2 (o que sobra)
**	potência

Divisão inteira e resto: *para que servem*

PROBLEMA	CONTA
O número é par?	<code>n % 2 == 0</code>
Último dígito	<code>n % 10</code>
Minutos em horas e minutos	<code>m // 60</code> e <code>m % 60</code>
Quantas caixas de 12 ovos	<code>ovos // 12</code> , sobram <code>ovos % 12</code>
É múltiplo de 3?	<code>n % 3 == 0</code>

par_impar.py · DIGITOU 2027

```
numero = int(input("Número: "))
print("É par?", numero % 2 == 0)
print("Último dígito:", numero % 10)
```

```
Número: 2027
É par? False
Último dígito: 7
```

`divmod(a, b)` devolve os dois de uma vez: `(a // b, a % b)` .

Quem vem *primeiro*

precedencia.py

```
print(2 + 3 * 4)      # 14: multiplicação antes da soma
print((2 + 3) * 4)    # 20: parênteses primeiro
print(-2 ** 2)        # -4: a potência vem antes do sinal
print((-2) ** 2)      # 4
print(10 - 4 - 3)     # 3: da esquerda para a direita
```

SAÍDA

```
14
20
-4
4
3
```

ORDEM	OPERADORES
1º	() parênteses
2º	** potência
3º	sinal: -x, +x
4º	* // %
5º	+ -
6º	comparações: == != < > in is
7º	not · depois and · por último or

Na dúvida, **use parênteses**: deixam a intenção clara para quem lê.

Comparações dão *True* ou *False*

comparacao.py

```
idade = 17
print(idade >= 18)      # False
print(idade == 17)      # True: == compara; = atribui
print(idade != 20)      # True
print(10 < idade < 20)   # True: comparação encadeada
print("a" < "b")        # True: ordem dos caracteres
```

SAÍDA

```
False
True
True
True
True
```

OPERADOR	PERGUNTA
==	É igual?
!=	É diferente?
> <	Maior? Menor?
>= <=	Maior ou igual? Menor ou igual?

`=` guarda, `==` compara. Trocar um pelo outro é o erro mais comum nesse assunto.

and, or e not

logicos.py

```
idade = 20
tem_ingresso = False
print(idade >= 18 and tem_ingresso)    # False: precisa dos dois
print(idade >= 18 or tem_ingresso)      # True: basta um
print(not tem_ingresso)                 # True: inverte
```

```
False
True
True
```

Esses resultados são a base das **condições** com **if** (aula 13): "se for maior de idade e tiver ingresso, entra".

A	B	A AND B	A OR B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Curto-circuito: no **and**, se o primeiro já é **False**, o segundo nem é avaliado; no **or**, se o primeiro é **True**, também não.

Atribuição *composta*

atribuicao.py

```
saldo = 100
saldo += 50    # saldo = saldo + 50
saldo -= 30    # saldo = saldo - 30
saldo *= 2     # saldo = saldo * 2
print(saldo)
x = y = 0      # mesmo valor para duas variáveis
a, b = 1, 2    # dois valores de uma vez
a, b = b, a    # troca os valores
print(x, y, a, b)
```

SAÍDA

```
240
0 0 2 1
```

✓ `x += 5` é o mesmo que `x = x + 5` ; vale para `-=` `*=` `/=` `//=` `%=` `**=`

✓ `100 + 50 - 30 = 120`, vezes 2 = **240**

✓ `a, b = b, a` troca dois valores sem variável auxiliar (as tuplas da aula 10 explicam por quê)

Python não tem `x++` nem `x--` . Use `x += 1` .

Pertinência *e identidade*

pertinencia.py

```
print("py" in "Python")      # False: diferencia maiúsculas
print("Py" in "Python")      # True
print(3 in [1, 2, 3])        # True
print("x" not in "Python")   # True
valor = None
print(valor is None)         # "is": só para None
```

SAÍDA

```
False
True
True
True
True
```

- ✓ `in` pergunta "está dentro?": em textos, listas, tuplas
- ✓ `not in` é o contrário
- ✓ `is` pergunta "é o mesmo objeto?", não "tem o mesmo valor?"

Regra prática do guia (cap. 3): para comparar valores, `==` ; `is` só com `None` (`x is None`).

Segundos em *h*, *min* e *s*

Enunciado: peça um total de segundos e mostre em horas, minutos e segundos, usando só `//` e `%`.

Uma resposta possível

```
# Exercício: segundos em horas, minutos e segundos
total = int(input("Segundos: "))
horas = total // 3600
minutos = total % 3600 // 60
segundos = total % 60
print(f"{horas}h {minutos}min {segundos}s")
```

SAÍDA (DIGITOU 7384)

Segundos: 7384
2h 3min 4s

- ✓ $7384 // 3600 = 2$ horas
- ✓ $7384 \% 3600 = 184$ s $\rightarrow 184 // 60 = 3$ minutos
- ✓ $7384 \% 60 = 4$ segundos
- ✓ Desafio: aceite também dias

Teste o que *ficou*

1. Quanto vale $17 \% 5$?

A) 3

B) 3.4

C) 2

D) 85

C. % é o resto.

2. Quanto vale $2 + 3 * 4$?

A) 20

B) 14

C) 24

D) 9

B. Multiplicação antes da soma.

3. True and False or True dá:

A) True

B) False

C) Erro

D) None

A. and primeiro: False or True.

4. Como somar 1 a x em Python?

A) x++

B) x += 1

C) x =+ 1

D) ++x

B. Não existe ++ em Python.

Contas, comparações *e decisões*

Aritméticos (`//` e `%` resolvem muitos problemas), precedência (na dúvida, parênteses), comparações que dão `True` / `False`, `and` / `or` / `not` com curto-circuito, `+=` no lugar de `++`, `in` para "está dentro" e `is` só para `None`.

`//` e `%`

Parênteses

`==` não é `=`

`and` · `or` · `not`

`+=`

`is None`

Próxima aula → 07 · Utilização de Strings

Fontes: docs.python.org/3/reference/expressions.html (operator precedence, comparisons, boolean operations); docs.python.org/3/library/stdtypes.html (numeric types), consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.